

Unity Inventory Framework Architecture

Unity Inventory Framework Architecture

A complete, modular, UI-agnostic inventory framework designed to serve as a foundation for any Unity project. This page captures the full architecture, philosophy, modules, and roadmap so you can revisit and expand it anytime.

Core Philosophy

A true framework must be:

- **UI-agnostic** - developers bring their own visuals.
- **Modular** - every part can be extended or replaced.
- **Predictable** - clean rules, consistent behavior.
- **Data-driven** - ScriptableObjects define items.
- **Event-driven** - UI and gameplay connect through events.
- **Genre-neutral** - works for RPG, survival, shooter, mobile, VR, etc.

The core handles **logic**. Developers handle **presentation**.

Core Modules

These are the foundational systems that must remain clean and stable.

ItemDefinition (ScriptableObject)

Holds all base item data:

- Name
- Icon reference (optional)
- Max stack size
- Category
- Tags
- Metadata dictionary (custom fields)

InventorySlot

Represents a single slot in an inventory:

- Item
- Quantity

- Runtime metadata
- Add/remove logic
- Stack checks

Inventory

The main logic container:

- Slots
- Capacity
- Weight rules (optional)
- Add/remove item methods
- Slot finding
- Inventory queries

InventoryEventBus

Event system for UI and gameplay:

- OnItemAdded
- OnItemRemoved
- OnStackChanged
- OnSlotChanged
- OnInventoryChanged
- OnInventoryFull

InventorySerializer

Handles persistence:

- JSON or binary
- Metadata serialization
- Versioning

Extension Points

These are the hooks that make the system a true framework.

Rule Blocks

Modular logic units developers can attach:

- CanAdd checks

- OnAdded
- OnRemoved

Examples:

- Spoilage
- Weight modifiers
- Slot restrictions
- Non-stackable rules

Metadata Channels

Custom fields developers can add:

- Durability
- Ammo count
- Quality
- Temperature
- Enchantments

Automatically serialized and exposed to UI.

Slot Types

Custom slot logic:

- Equipment slots
- Hotbar slots
- Restricted slots
- Weight-limited slots

Optional Modules (Future Expansions)

These can be added later as updates or separate assets.

Equipment Module

- EquipmentSlot
- Stat modifiers
- Equip/unequip rules

Crafting Module

- Recipe definitions

- Crafting stations
- Time-based crafting

Hotbar Module

- Quick-use slots
- Cooldowns
- Activation events

Loot Module

- Loot tables
- Weighted rarity
- Procedural item generation

Multiplayer Sync Module

- Server-authoritative logic
- Delta updates
- Ownership rules

Innovation Layer

Features that differentiate this framework from existing assets.

UI-Binding Wizard

Inspector tool that:

- Detects UI elements
- Lets developers bind them to events
- Auto-generates boilerplate code

Smart Stacking Rules

Configurable stacking behavior:

- Merge
- Overflow
- Reject
- Convert (e.g., 10 wood → 1 plank)

Inventory Profiles

Pre-configured rule sets:

- Survival
- RPG
- Shooter
- Mobile

Developers can switch profiles instantly.

Development Roadmap

A clear sequence for building the framework.

Milestone 1 - Core Engine

- ItemDefinition
- InventorySlot
- Inventory
- EventBus
- Basic serialization

Milestone 2 - Extension System

- RuleBlocks
- Metadata channels
- Slot types

Milestone 3 - UI-Binding Wizard

- Inspector tool
- Auto-binding logic

This page serves as the persistent reference for the Unity Inventory Framework. You can revisit it anytime to expand, refine, or build upon the architecture.